Université de Caen Normandie — Licence 3 Informatique

Conception logicielle 2 — Année 2024–2025



RAPPORT DE PROJET

Jeu de Hex et bandits manchots

Réalisé par

Kahina AKKACHE

Ahmed Bezeid Mohamed EL MAMY

Imen BOUKRAA

Safa BAALY

Encadrant

M. Gregory BONNET

Table des matières

1 Introduction **3**

1.1 Description générale du projet . . . 3

1.2 Présentation du plan du rapport . . . 3

2 Objectifs du projet **4**

2.1 Problématique . . . 4

2.2 Approche et modélisation . . . 4

2.3 Travaux connexes . . . 5

3 Fonctionnalités implémentées **5**

3.1 Description des fonctionnalités . . . 5

3.2 Répartition des tâches . . . 6

4 Éléments techniques **6**

4.1 Détection de victoire par DFS . . . 6

4.2 Monte Carlo Tree Search (MCTS) . . . 6

4.2.1 Principes généraux . . . 6

4.2.2 Les quatre phases . . . 6

4.2.3 La formule UCT . . . 7

4.3 Rapid Action Value Estimation (RAVE) . . . 8

4.3.1 Motivation et principe . . . 8

4.3.2 Statistiques maintenues . . . 8

4.3.3 Formule de sélection RAVE . . . 8

4.3.4 Avantages de RAVE par rapport à MCTS . . . 8

5 Architecture du projet **8**

5.1 Organisation en packages . . . 9

5.2 Diagrammes de classes . . . 9

5.2.1 Le package `Game` . . . 9

5.2.2 Le package `Mcts` . . . 10

5.2.3 Le package `Rave` . . . 11

6 Expérimentations **12**

6.1 Problématique et conditions expérimentales . . . 12

6.2 Analyse des expérimentations . . . 13

6.2.1 MCTS vs MCTS . . . 13

6.2.2 RAVE vs RAVE . . . 17
6.2.3 MCTS vs RAVE . . . 21
6.2.4 RAVE vs MCTS . . . 27
6.2.5 Conclusion de l'analyse expérimentale . . . 32
6.3 Critère de sélection du meilleur enfant . . . 32
6.4 Paramètre d'exploration c . . . 32

7 Reverse Hex (Misère Hex) 32
7.1 Présentation de la variante . . . 32
7.2 Adaptations techniques . . . 33
7.2.1 Modification de la condition de victoire . . . 33
7.2.2 Modification de la simulation dans MCTS et RAVE . . . 33
7.2.3 Résumé des modifications . . . 33
7.3 Résultats de test . . . 34
7.4 Analyse et intérêt de la variante . . . 34

8 Conclusion 34

Références 35

1. Introduction

1.1 Description générale du projet

Ce rapport présente le travail que nous avons réalisé dans le cadre de notre Licence 3 en Informatique à l'Université de Caen Normandie. Il décrit l'implémentation du jeu de Hex, un jeu de stratégie combinatoire où deux joueurs s'affrontent sur un plateau hexagonal en cherchant à relier leurs deux bords opposés. L'objectif de ce projet est de faire jouer des robots de façon intelligente en utilisant l'algorithme Monte Carlo Tree Search (MCTS) ainsi que son optimisation RAVE, et d'étudier l'influence du budget de simulation sur les résultats des parties. Le projet inclut également une variante du jeu appelée **Reverse Hex** (ou Misère Hex), dans laquelle la règle de victoire est inversée : le joueur qui relie ses deux bords opposés perd la partie.

Le jeu de Hex est un jeu abstrait inventé par Piet Hein en 1942 et redécouvert indépendamment par John Nash en 1948. La taille standard du plateau est de 11×11, mais dans notre projet nous avons utilisé trois tailles : 4×4, 8×8 et 10×10, afin de mener nos expériences dans un temps raisonnable.

Chaque joueur est associé à une couleur :

- **Rouge** : doit relier le bord Nord au bord Sud du plateau.
- **Bleu** : doit relier le bord Ouest au bord Est.

Les joueurs jouent chacun leur tour en plaçant un pion de leur couleur sur une case vide. Une fois posé, un pion ne peut plus être déplacé ni retiré.

Le jeu assure toujours une victoire : il ne peut pas y avoir de match nul. Une fois toutes les cases occupées, au moins l'un des deux joueurs aura forcément relié ses bords opposés. Le théorème de Nash démontre que le premier joueur a une stratégie gagnante, bien que cette stratégie optimale soit impossible à identifier en pratique sur les grands plateaux.

Hex est également reconnu pour son intérêt mathématique, notamment en théorie des jeux et en topologie, car il est directement lié au théorème du cheminement.

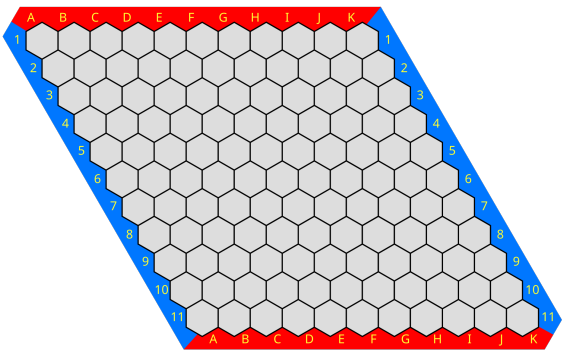


FIGURE 1.1. Plateau au début de la partie.

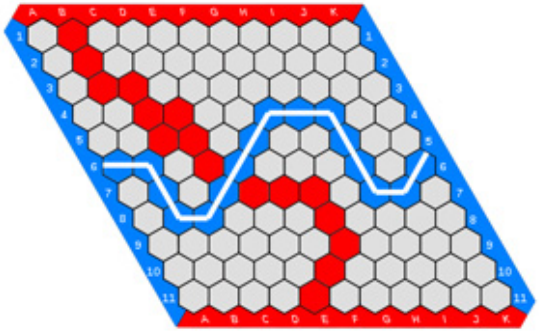


FIGURE 1.2. Plateau à la fin de la partie.

1.2 Présentation du plan du rapport

Ce rapport est organisé de la façon suivante :

- **Objectifs du projet** : présentation de la problématique et de la modélisation choisie pour repré-

senter le jeu.

- **Fonctionnalités implémentées** : description des fonctionnalités développées et répartition des tâches entre les membres du groupe.
- **Éléments techniques** : explication des algorithmes utilisés (DFS, MCTS, RAVE) et des structures de données associées.
- **Architecture du projet** : organisation du code en packages et interactions entre les classes.
- **Expérimentations** : analyse des résultats selon le budget, la taille du plateau et les algorithmes utilisés.
- **Reverse Hex** : présentation de la variante Misère Hex, adaptations techniques apportées au code et résultats expérimentaux.
- **Conclusion** : bilan du projet et pistes d'amélioration.

2. Objectifs du projet

2.1 Problématique

L'objectif principal de ce projet est d'étudier l'influence du rapport de budget entre les deux joueurs sur leur probabilité de victoire, en fonction de la taille du plateau. Plus précisément, nous cherchons à répondre aux questions suivantes :

1. Existe-t-il un seuil de budget à partir duquel un joueur devient significativement avantagé par rapport à l'autre ?
2. Comment la taille du plateau influence-t-elle la relation entre le budget et les résultats des parties ?

Le terme **budget** désigne ici le nombre de simulations qu'un joueur peut effectuer avant de choisir son coup. Plus ce budget est élevé, plus l'algorithme explore de possibilités et prend de meilleures décisions. Un joueur avec un budget plus grand a donc en théorie un avantage sur son adversaire, mais cet avantage peut être contrebalancé par le fait de jouer en premier.

2.2 Approche et modélisation

Pour représenter le plateau de jeu, nous avons utilisé une structure de graphe où chaque case hexagonale correspond à un nœud et chaque connexion entre deux cases adjacentes correspond à une arête. Cette représentation nous a permis d'appliquer un algorithme de parcours en profondeur (DFS) pour vérifier si un joueur a réussi à relier ses deux bords opposés, en explorant toutes les cases connectées de même couleur.

Pour l'intelligence artificielle, nous avons implémenté l'algorithme Monte Carlo Tree Search (MCTS). C'est un algorithme basé sur des simulations aléatoires de parties entières, qui construit progressivement un arbre de recherche où chaque nœud représente un état du jeu. Il suit quatre étapes à chaque itération : la sélection, l'expansion, la simulation et la rétropropagation. L'un de ses avantages est qu'il ne nécessite aucune connaissance préalable sur le jeu, ce qui le rend facile à adapter à différents contextes.

Nous avons ensuite amélioré MCTS avec l'optimisation RAVE. L'idée est d'utiliser les informations collectées durant toutes les simulations pour évaluer un coup, et pas seulement celles de la branche explorée. RAVE combine deux sources d'information : la valeur classique de MCTS, basée sur les parties jouées dans une branche précise, et la valeur RAVE, basée sur toutes les simulations où ce coup a été joué. Cette combinaison permet à l'algorithme de converger plus rapidement vers de bonnes

décisions.

Pour lancer les expériences, nous avons créé un script shell `ExpRave.sh` qui compile le projet et lance automatiquement les quatre configurations suivantes :

- **MCTS vs MCTS** : les deux joueurs utilisent MCTS.
- **RAVE vs RAVE** : les deux joueurs utilisent RAVE.
- **MCTS vs RAVE** : le premier joueur utilise MCTS, le second RAVE.
- **RAVE vs MCTS** : le premier joueur utilise RAVE, le second MCTS.

Les résultats de chaque partie sont enregistrés automatiquement dans des fichiers de logs, contenant pour chaque expérience : la taille de la grille, le premier joueur, le gagnant, le budget et l'algorithme utilisé par chaque joueur. Ces données nous ont ensuite permis d'analyser l'impact du budget et de la taille du plateau sur les résultats.

2.3 Travaux connexes

Le jeu de Hex a fait l'objet de plusieurs travaux de recherche en intelligence artificielle. Bien que ses règles soient simples, sa complexité combinatoire en fait un bon terrain pour tester et comparer des algorithmes de recherche. Plusieurs compétitions entre programmes ont été organisées, ce qui a poussé les chercheurs à améliorer continuellement leurs agents.

Le programme **MoHex**, développé par l'Université d'Alberta, est l'un des agents les plus connus pour le jeu de Hex. Il est basé sur MCTS avec plusieurs optimisations, et a remporté de nombreuses compétitions internationales. MoHex montre qu'avec un bon budget de simulations et des améliorations bien choisies, MCTS peut atteindre un très bon niveau de jeu sur le Hex. Notre projet s'inspire de cette approche, mais à une échelle plus petite et avec un focus sur l'impact du budget.

L'optimisation RAVE que nous utilisons vient à l'origine du programme MoGo, développé pour le jeu de Go. C'est dans ce contexte que Gelly et Silver ont proposé RAVE en 2007, pour accélérer la convergence de MCTS en réutilisant les informations de toutes les simulations. Cette idée a ensuite été reprise dans beaucoup d'autres jeux, dont le Hex, car elle fonctionne particulièrement bien dans les jeux où un bon coup reste bon quelle que soit la position exacte où il est joué.

3. Fonctionnalités implémentées

3.1 Description des fonctionnalités

Nous avons développé plusieurs fonctionnalités pour permettre différents modes de jeu et faciliter l'analyse des résultats :

1. **Mode robot vs robot** : deux agents automatiques s'affrontent en utilisant MCTS ou RAVE. C'est le mode principal de notre projet, qui nous a permis de lancer un grand nombre de parties automatiquement et de collecter des données pour nos expériences.

3.2 Répartition des tâches

Membre	Tâches réalisées
Ahmed Bezeid Mohamed El Mamy	Implémentation du Reverse Hex et implémentation de RAVE
Kahina Akkache	Implémentation de la logique du jeu, de MCTS et expérimentations
Imen Boukraa	Implémentation de MCTS et rédaction du rapport
Safa Baaly	Implémentation de RAVE et rédaction du rapport

4. Éléments techniques

4.1 Détection de victoire par DFS

La détection de victoire repose sur un parcours en profondeur (DFS). Pour vérifier si **Rouge** a gagné, on part de toutes les cases rouges situées sur le bord Nord et on explore récursivement toutes les cases rouges adjacentes. Si l'exploration atteint le bord Sud, **Rouge** a gagné. Le même principe s'applique à **Bleu** entre les bords Ouest et Est.

Propriétés du DFS pour Hex

- Complexité en $O(n^2)$ pour un plateau $n \times n$.
- Pas de faux positifs ni de faux négatifs : dès qu'un joueur relie ses bords, le DFS le détecte immédiatement.
- Appelé après chaque coup posé pour terminer la partie dès que la victoire est atteinte.

4.2 Monte Carlo Tree Search (MCTS)

4.2.1 Principes généraux

Le Monte Carlo Tree Search est un algorithme de recherche heuristique utilisé dans les jeux à deux joueurs à information parfaite. Contrairement aux approches minimax classiques qui explorent exhaustivement l'arbre de jeu, MCTS guide la recherche par des simulations aléatoires (*playouts*) et concentre progressivement ses efforts sur les coups les plus prometteurs.

L'algorithme maintient un arbre dont chaque nœud représente un état du jeu, et chaque arc une action possible. À chaque nœud sont associées deux statistiques : le nombre de victoires w_i et le nombre total de visites n_i.

4.2.2 Les quatre phases

MCTS répète un cycle de quatre phases jusqu'à épuisement du budget de simulation :

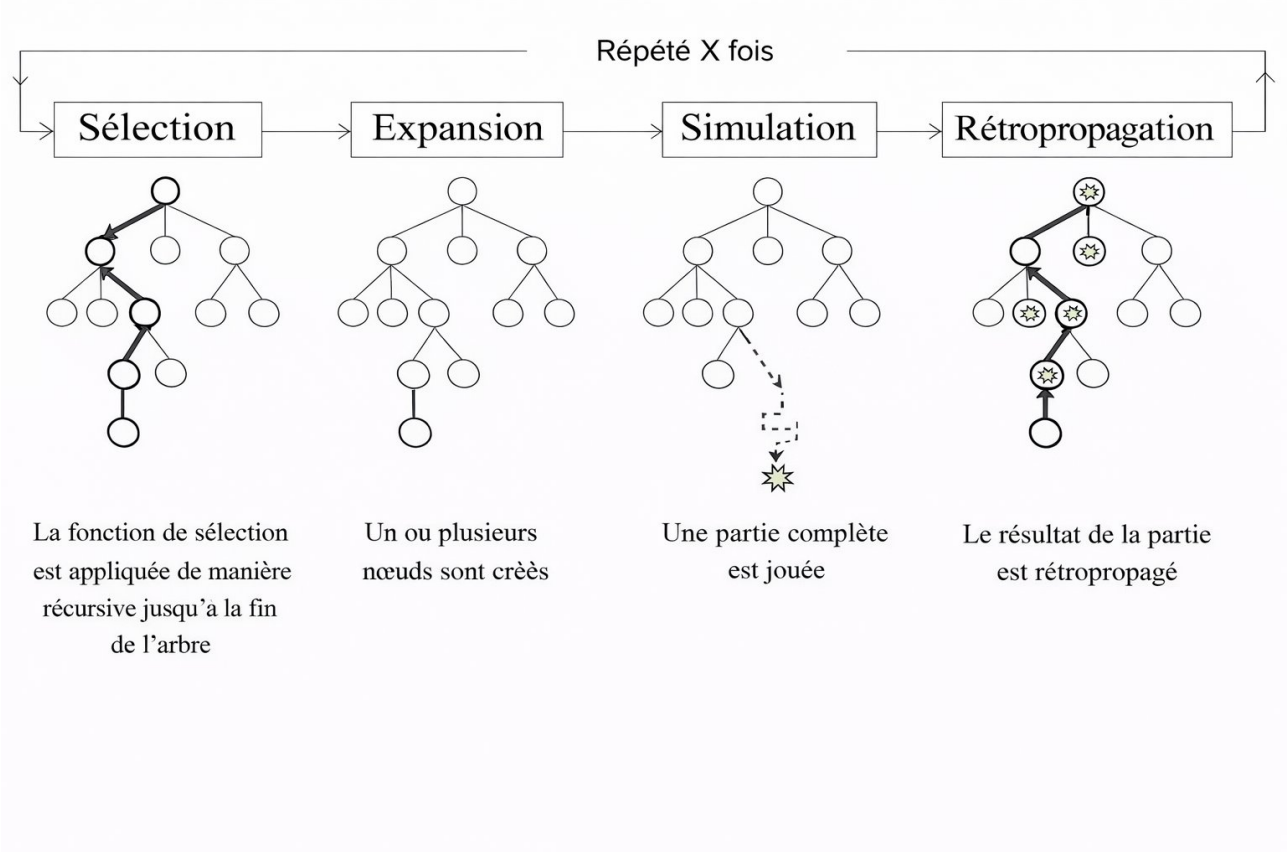


FIGURE 4.1. Les quatre phases de l'algorithme MCTS répétées à chaque itération.

Les 4 phases de MCTS

1. **Sélection** : depuis la racine, descendre dans l'arbre en appliquant récursivement la politique UCT jusqu'à un nœud feuille.
2. **Expansion** : créer un ou plusieurs nœuds enfants à partir du nœud feuille atteint.
3. **Simulation** : jouer une partie aléatoire complète depuis le nœud nouvellement créé jusqu'à un état terminal.
4. **Rétropropagation** : remonter le résultat de la simulation dans tous les nœuds ancêtres du nœud visité.

4.2.3 La formule UCT

La phase de sélection repose sur la formule **UCT** (*Upper Confidence Bound for Trees*), qui équilibre l'exploitation des nœuds déjà bien évalués et l'exploration des nœuds peu visités :

Formule UCT

$$\text{UCT}(i) = \underbrace{\frac{w_i}{n_i}}_{\text{exploitation}} + c \times \underbrace{\sqrt{\frac{\ln(n_p)}{n_i}}}_{\text{exploration}}$$

- w_i : nombre de victoires du nœud i
- n_i : nombre de visites du nœud i
- n_p : nombre de visites du nœud parent
- c : paramètre d'exploration (typiquement $c = \sqrt{2}$)

Le paramètre c contrôle le compromis exploration/exploitation : une valeur élevée favorise l'exploration, une valeur faible favorise l'exploitation des meilleurs coups connus.

4.3 Rapid Action Value Estimation (RAVE)

4.3.1 Motivation et principe

MCTS présente une limitation dans les premières phases de recherche : les nœuds peu visités ont des estimations peu fiables. L'algorithme RAVE remédie à ce problème en exploitant une hypothèse propre au jeu de Hex : *un coup joué à une certaine position tend à avoir la même valeur quelle que soit l'étape de la partie où il est joué*. Cette hypothèse est connue sous le nom **AMAF** (*All Moves As First*).

Chaque simulation met à jour non seulement le nœud visité, mais aussi tous les nœuds correspondant aux coups joués durant cette simulation. Cela permet de propager des statistiques beaucoup plus rapidement à travers l'arbre.

4.3.2 Statistiques maintenues

À chaque nœud, RAVE maintient deux compteurs supplémentaires :

- `raveVictories` : victoires dans les simulations où ce coup a été joué (à n'importe quel moment).
- `raveVisits` : nombre total de simulations où ce coup est apparu.

4.3.3 Formule de sélection RAVE

La politique de sélection RAVE combine la valeur MCTS classique et la valeur AMAF via un coefficient de mélange β :

Formule de sélection RAVE

$$Q_{\text{RAVE}} = (1-\beta)\,\frac{w_i}{n_i} + \beta\,\frac{\mathit{raveVictories}}{\mathit{raveVisits}} + C\sqrt{\frac{\ln(n_p)}{n_i}}$$

$$\beta = \sqrt{\frac{k}{3\,n_i + k}}, \qquad C = \sqrt{2}$$

Le coefficient β décroît à mesure que n_i augmente : l'influence de la valeur AMAF est forte au début (peu de visites) et s'efface progressivement en faveur de la valeur MCTS plus précise.

Le paramètre k contrôle la vitesse de cette transition. Une valeur élevée de k maintient l'influence de l'heuristique AMAF plus longtemps, même lorsque le nœud a été visité de nombreuses fois. À l'inverse, une valeur faible de k accélère le passage vers la valeur MCTS pure. Dans notre implémentation, $k = 300$ représente un bon compromis : AMAF domine jusqu'à environ 100 visites par nœud, puis MCTS reprend progressivement la main.

4.3.4 Avantages de RAVE par rapport à MCTS

- **Apprentissage plus rapide** : les statistiques AMAF permettent d'évaluer des coups non encore explorés directement dans l'arbre.
- **Meilleures décisions à faible budget** : pour de petits budgets, RAVE tend à produire de meilleures décisions que MCTS pur.
- **Adaptabilité** : le coefficient β assure une transition douce entre l'heuristique AMAF et l'estimation MCTS.

5. Architecture du projet

5.1 Organisation en packages

Le projet est structuré en trois packages Java distincts, chacun regroupant les classes liées à une responsabilité précise :

- **Game** : contient la logique du jeu, la représentation du plateau, les joueurs, la détection de victoire et l'orchestration d'une partie.
- **Mcts** : contient l'implémentation de l'algorithme Monte Carlo Tree Search, avec la gestion de l'arbre de recherche, des états et du cache.
- **Rave** : contient l'implémentation de l'optimisation RAVE, avec les statistiques AMAF, la formule de sélection combinée et le résultat de simulation.

Les packages `Mcts` et `Rave` dépendent tous deux du package `Game` pour accéder aux classes `Board`, `Color`, `Position` et `DFS`. De plus, `Rave` réutilise directement les classes `State` et `CacheMap` définies dans `Mcts`.

5.2 Diagrammes de classes

5.2.1 Le package `Game`

Le package `Game` est responsable de la gestion de la logique et des règles du jeu. Il regroupe toutes les classes nécessaires à la représentation du plateau, des joueurs et au déroulement d'une partie.

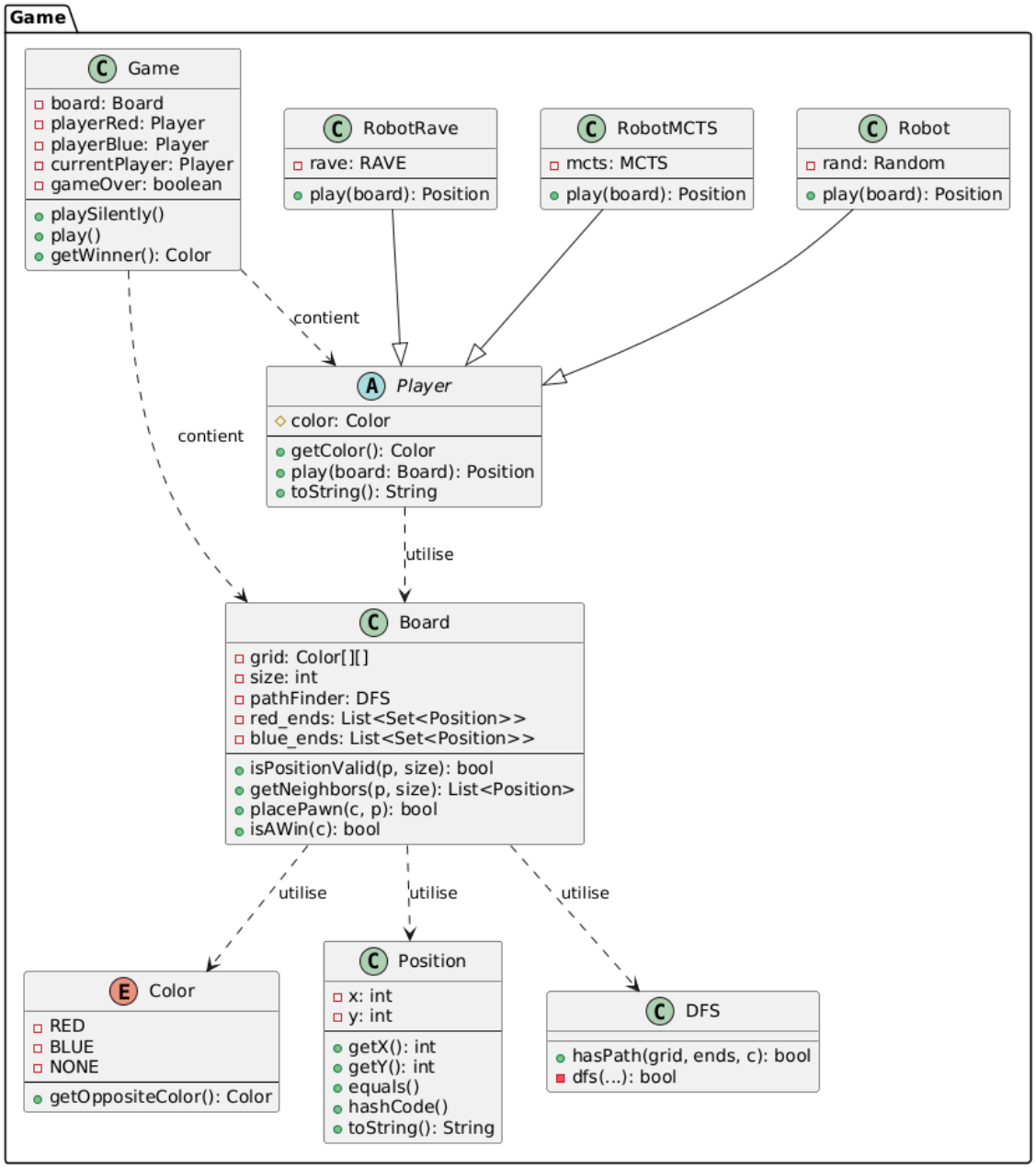


FIGURE 5.1. Diagramme de classes du package `Game`.

5.2.2 Le package `Mcts`

Le package `Mcts` est dédié à l'implémentation de l'algorithme Monte Carlo Tree Search. Il regroupe les classes nécessaires à la gestion de l'arbre de recherche, à la représentation des états et à l'optimisation mémoire.

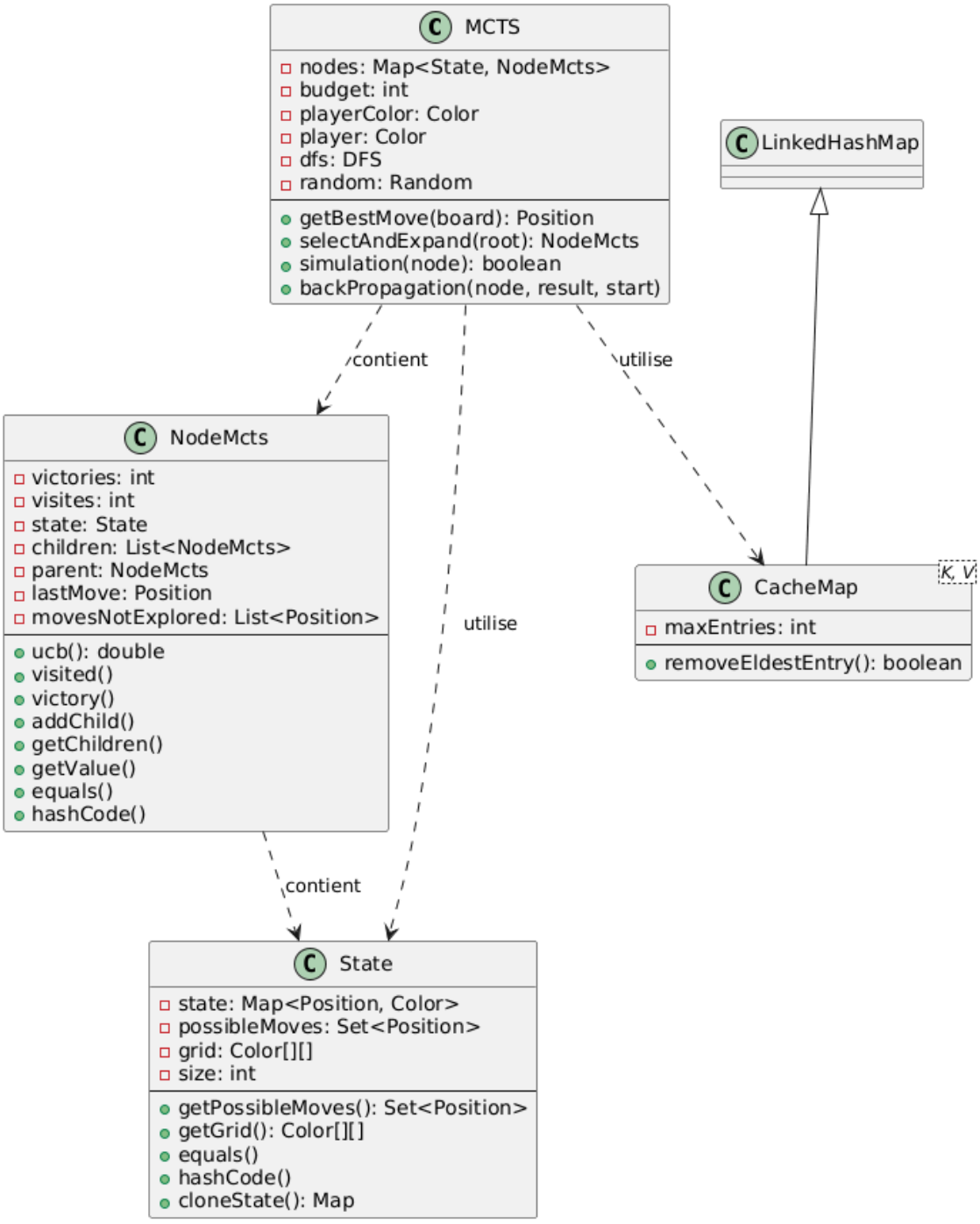


FIGURE 5.2. Diagramme de classes du package `Mcts`.

5.2.3 Le package `Rave`

Le package `Rave` est dédié à l'implémentation de l'optimisation RAVE (*Rapid Action Value Estimation*). Il étend l'approche MCTS en maintenant des statistiques AMAF supplémentaires à chaque nœud et en réutilisant `State` et `CacheMap` du package `Mcts`.

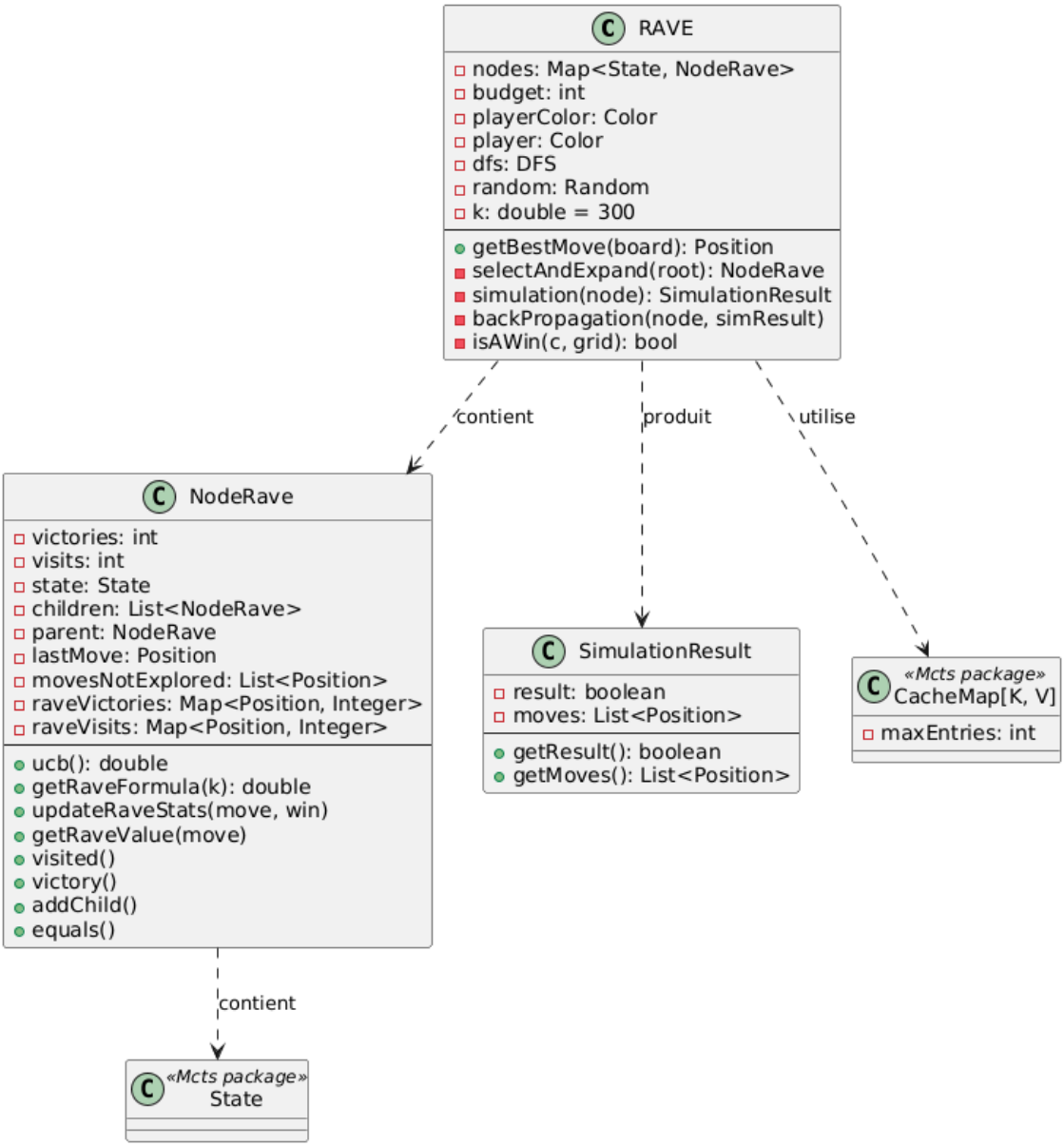


FIGURE 5.3. Diagramme de classes du package `Rave`.

6. Expérimentations

6.1 Problématique et conditions expérimentales

Question de recherche

Quelle influence le rapport de budget entre les deux joueurs a-t-il sur leur probabilité de victoire en fonction de la taille de la grille ?

Les conditions de test sont les suivantes :

Paramètre	Valeurs testées
Joueur qui commence	Rouge (premier joueur)
Taille de la grille	4×4, 8×8, 10×10
Budget de simulation	De $2^0 = 1$ à $2^{15} = 32\,768$
Rapports de budget	1:1, 1:2, 1:5, 2:1, 5:1
Confrontations	MCTS/MCTS, RAVE/RAVE, MCTS/RAVE, RAVE/MCTS

Le rapport $r{:}b$ signifie que **Rouge** dispose d'un budget r fois celui de **Bleu**. Par exemple, $5{:}1$ indique que **Rouge** dispose de 5 fois plus de simulations.

6.2 Analyse des expérimentations

6.2.1 MCTS vs MCTS

Comportements communs aux trois tailles

Trois comportements se retrouvent systématiquement sur les trois graphiques MCTS vs MCTS. Les comprendre d'abord permet d'isoler ce qui distingue chaque taille dans la section suivante.

Le ratio budgétaire gouverne la hiérarchie des courbes

■ Ce qu'on observe	■ Pourquoi
Sur les trois graphiques, la hiérarchie est identique : courbes 5 :1 et 2 :1 toujours en haut, courbes 1 :5 et 1 :2 toujours en bas, courbe 1 :1 au milieu. Cette hiérarchie est stable quelle que soit la taille.	MCTS explore l'arbre par simulation aléatoire. Le joueur avec plus de sim/coup voit plus profondément l'arbre et prend de meilleures décisions. Sur Hex, une erreur tactique non détectée se paye plusieurs coups plus tard — d'où l'impact direct et immédiat du ratio sur le résultat final.

La courbe 1 :1 révèle l'avantage du premier joueur, atténué par la taille

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 se situe au-dessus de 50% sur les trois graphiques, mais son niveau dépend de la taille : bien au-dessus sur taille 4, à la limite de significativité sur taille 8, et dans le bruit statistique sur taille 10.	Quand les deux joueurs ont le même budget, seul l'avantage du premier joueur fait la différence. Sur Hex, jouer en premier est un avantage théorique prouvé. Mais plus le plateau est grand, plus les parties sont longues et plus le premier coup représente une fraction infime de la décision globale — l'avantage se noie dans la complexité.

La symétrie des courbes inverses valide le protocole

■ Ce qu'on observe	■ Pourquoi
Sur les trois tailles, les courbes 2 :1 et 1 :2 sont quasi-symétriques autour de 50%, idem pour 5 :1 et 1 :5.	Inverser les budgets RED/BLUE inverse exactement les rôles. Cette symétrie confirme la cohérence du protocole expérimental.

Point clé

Synthèse : Le ratio budgétaire est le facteur dominant, mais l'avantage du premier joueur reste visible en ratio 1 :1. Plus la taille augmente, plus les effets du ratio se déclenchent tôt, plus ils sont irréversibles, et plus l'avantage du premier joueur se dilue.

Spécificités de chaque taille

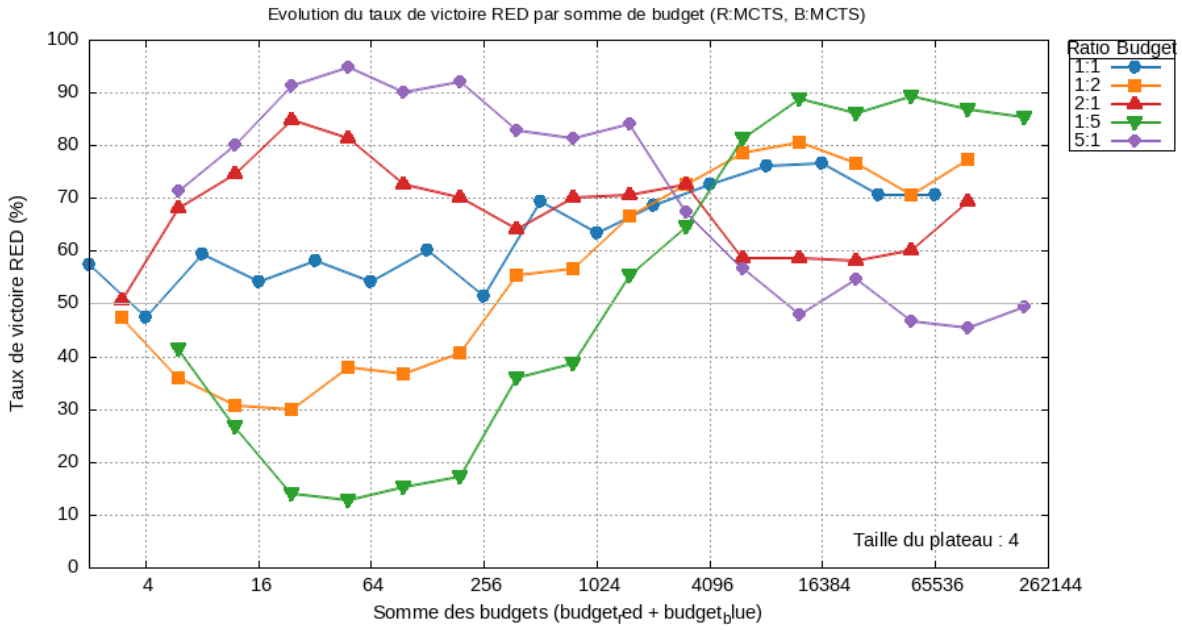


FIGURE 6.1. Taux de victoire RED · MCTS vs MCTS · Plateau 4×4 · 150 parties/point

Plateau 4×4 — 16 cases. Le plateau 4×4 comporte 16 cases, avec des parties de 8 à 12 coups en moyenne. C'est la seule configuration où MCTS peut progressivement quasi-résoudre le jeu aux très grands budgets.

Ratio 1 :1 — L'avantage du premier joueur bien visible et stable

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 se maintient entre 55% et 75% sur toute la plage, de façon stable et au-dessus de l'IC95 ±8 pts — signal significatif.	À budget égal, les deux joueurs ont la même puissance de calcul. La différence vient uniquement de l'avantage du premier joueur : sur 16 cases, chaque coup impacte fortement la connectivité. Cet avantage reste stable quel que soit le budget.

Ratios 2 :1 et 5 :1 — Comportement en cloche : domination puis atténuation

■ Ce qu'on observe	■ Pourquoi
Les courbes 2 :1 et 5 :1 montent vers 85–95% aux budgets intermédiaires, puis redescendent vers 70–75% aux très grands budgets.	La montée : RED explore mieux l'arbre et exploite les erreurs de BLUE. La redescente est la signature de la quasi-résolution : aux très grands budgets, même BLUE dispose de suffisamment de sim/coup pour explorer l'intégralité du petit plateau.

Ratios 1 :2 et 1 :5 — Effondrement puis récupération : l'avantage de RED comme filet

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :5 chute vers 10–15% aux budgets intermédiaires puis remonte vers 35–40% aux très grands budgets.	La remontée tient à la quasi-résolution du plateau 4×4 et à l'avantage du premier joueur qui agit comme un filet, empêchant les courbes défavorables de tomber à 0% et permettant une remontée partielle. Sur tailles 8 et 10, ni la quasi-résolution ni ce filet n'existent.

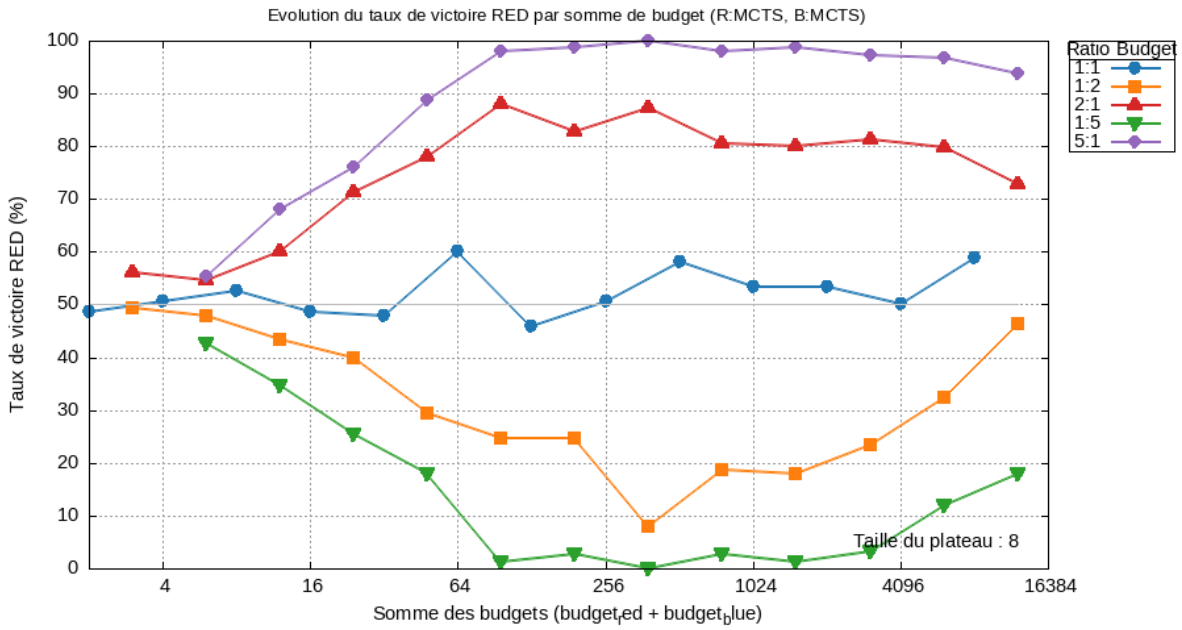


FIGURE 6.2. Taux de victoire RED · MCTS vs MCTS · Plateau 8×8 · 150 parties/point

Plateau 8×8 — 64 cases. Ratio 1 :1 — L'avantage du premier joueur s'efface

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 reste plate entre 47% et 60%, sans tendance claire. Les fluctuations entrent dans l'IC95 ±8 pts.	Sur 64 cases, le premier coup représente moins d'1/30 de la décision globale. L'avantage du premier joueur existe toujours en théorie mais est statistiquement non significatif à cette taille.

Ratios 2 :1 et 5 :1 — Monotonie stricte sans redescente

■ Ce qu'on observe	■ Pourquoi
Les courbes 2 :1 et 5 :1 montent de façon quasi-monotone sans jamais redescendre. La courbe 5 :1 atteint 95–100% dès les budgets intermédiaires et s'y maintient jusqu'à la fin de la plage testée.	Sur 64 cases, aucun budget testé ne permet à BLUE de quasi-résoudre le jeu. L'avantage de ratio s'accumule indéfiniment.

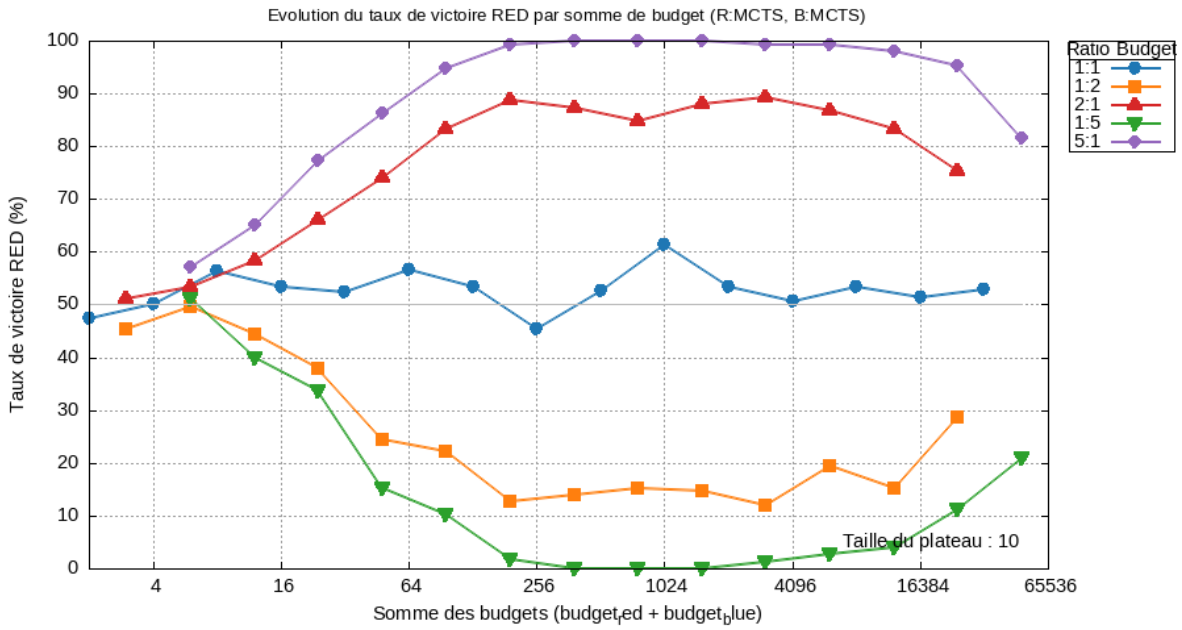


FIGURE 6.3. Taux de victoire RED · MCTS vs MCTS · Plateau 10×10 · 150 parties/point

Plateau 10×10 — 100 cases. Ratios défavorables — Un effondrement encore plus précoce, aucun filet

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :5 atteint 0% encore plus tôt qu'en taille 8 et reste à 0% sur toute la plage testée, sans aucune remontée.	Sur 100 cases, manquer une tactique au coup 10 laisse 70 coups pour que BLUE consolide son avantage. L'avantage du premier joueur de RED est totalement écrasé.

6.2.2 RAVE vs RAVE

Comportements communs aux trois tailles

RAVE enrichit MCTS avec l'heuristique AMAF : chaque coup joué lors d'une simulation met à jour les statistiques de ce coup dans tous les nœuds ancêtres. Avec $k = 300$, AMAF domine jusqu'à ∼100 visites/nœud puis MCTS reprend la main.

■ Ce qu'on observe	■ Pourquoi
À ratio identique, les courbes RAVE atteignent leurs niveaux de performance à des budgets nettement plus faibles que MCTS. Les mêmes taux de victoire sont obtenus avec bien moins de simulations.	AMAF transforme chaque simulation en information sur tous les coups joués, pas seulement sur les branches directement visitées. RAVE extrait donc plus d'information par simulation que MCTS. À petit budget, les statistiques AMAF sont trop pauvres — RAVE se comporte comme MCTS. À très grand budget, β tend vers 0 et les deux convergent. C'est à budget intermédiaire que l'avantage de RAVE est maximal.

■ Ce qu'on observe	■ Pourquoi
Avec RAVE, la courbe 1 :1 est systématiquement plus favorable à RED qu'avec MCTS, sur toutes les tailles. Sur taille 10, RAVE révèle un avantage du premier joueur que MCTS ne pouvait plus mesurer.	Les coups d'ouverture de RED apparaissent dans toutes les simulations partant de la racine et accumulent bien plus de statistiques AMAF que les coups de BLUE. RAVE sur-évalue donc les coups d'ouverture de RED. Cet effet est d'autant plus fort que les simulations sont longues — ce qui explique pourquoi il est plus marqué sur les grands plateaux.

■ Ce qu'on observe	■ Pourquoi
Sur les trois graphiques : une zone de petits budgets où toutes les courbes sont groupées, une zone intermédiaire où les écarts sont maximaux, et une zone de grands budgets où les courbes extrêmes se resserrent légèrement.	Phase 1 : AMAF trop peu alimenté — RAVE se comporte comme MCTS. Phase 2 : AMAF fiable et actif — amplification maximale des écarts. Phase 3 : β tend vers 0, MCTS reprend la main, correction progressive des biais AMAF.

Point clé

Synthèse : RAVE amplifie les trois facteurs simultanément — le ratio budgétaire, la taille du plateau, et l'avantage du premier joueur. Son efficacité supérieure par simulation explique pourquoi tous ces effets se déclenchent plus tôt et plus intensément qu'avec MCTS.

Spécificités de chaque taille

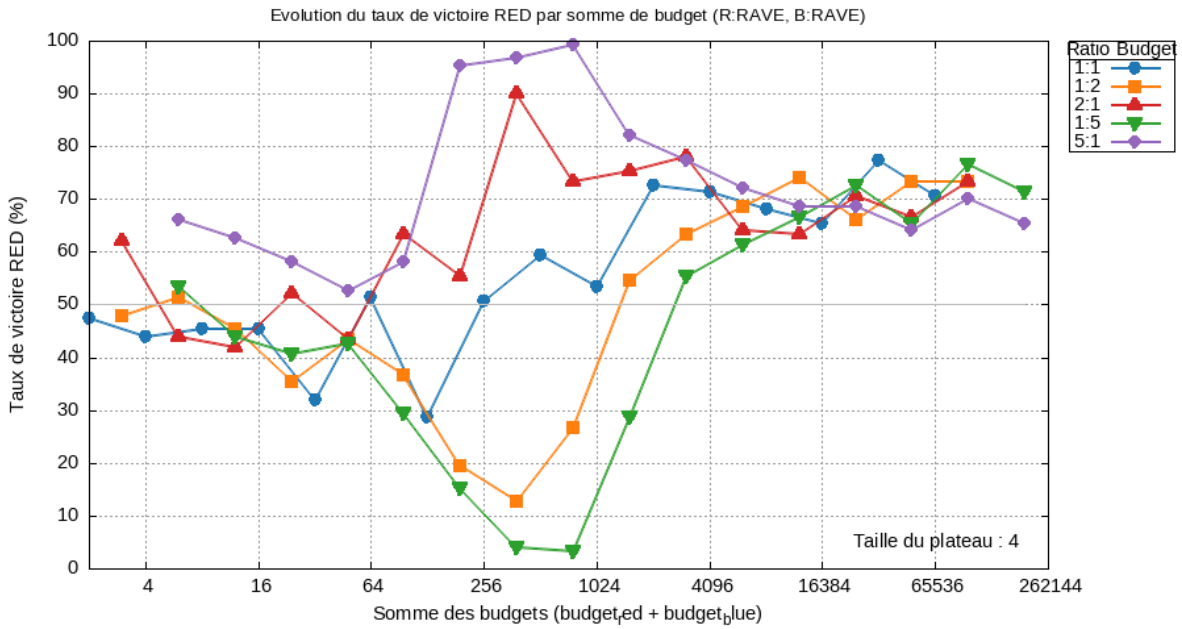


FIGURE 6.4. Taux de victoire RED · RAVE vs RAVE · Plateau 4×4 · 150 parties/point

Plateau 4×4 — 16 cases. Sur 16 cases, chaque simulation ne couvre que ∼16 coups : les statistiques AMAF sont très pauvres et les comportements instables.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 monte régulièrement de ∼47% à ∼73% sur toute la plage. C'est la seule courbe 1 :1 croissante de toute l'expérience — avec MCTS elle était stable autour de 60–65%.	Avec MCTS, la courbe 1 :1 était stable car aucun mécanisme ne la faisait croître avec le budget. Avec RAVE, le biais AMAF sur les coups d'ouverture de RED s'accentue à mesure que le budget augmente : les statistiques AMAF de RED deviennent de plus en plus précises et orientent fortement la sélection en sa faveur. RAVE amplifie donc l'avantage du premier joueur de façon croissante.
La courbe 1 :5 oscille entre ∼4% et ∼73% sans pattern identifiable.	RED a très peu de sim/coup → statistiques AMAF quasi-inexistantes malgré un β élevé. Le terme UCB additionnel crée une sur-exploration chaotique.

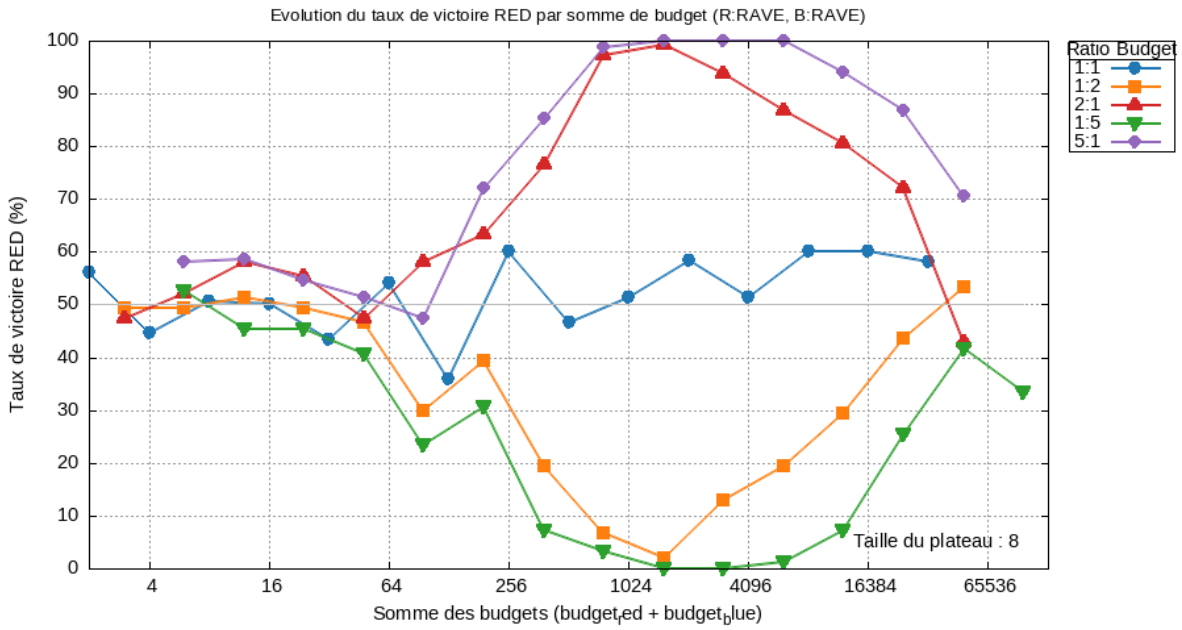


FIGURE 6.5. Taux de victoire RED · RAVE vs RAVE · Plateau 8×8 · 150 parties/point

Plateau 8×8 — 64 cases. Le plateau 8×8 est la configuration où RAVE révèle le mieux ses propriétés. Les trois phases comportementales sont clairement visibles et l'efficacité supérieure de RAVE par rapport à MCTS est particulièrement lisible.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 forme un dôme : elle monte vers ∼58–60% aux budgets intermédiaires, puis redescend vers ∼50% aux grands budgets. Avec MCTS taille 8, la courbe était plate et non significative.	RAVE révèle temporairement l'avantage du premier joueur que MCTS ne pouvait plus mesurer sur 64 cases. Le biais AMAF sur les coups d'ouverture de RED se manifeste quand les statistiques sont fiables aux budgets intermédiaires. Aux grands budgets, β tend vers 0 et le biais disparaît — la courbe revient vers 50%.
Les courbes 5 :1 et 2 :1 montent brutalement vers 97–100% et 80–90% à des budgets où MCTS n'atteignait que ∼65–70%. RAVE atteint donc le même niveau de performance avec bien moins de simulations que MCTS.	C'est la fenêtre optimale de RAVE : AMAF est fiable (assez de sim/coup pour couvrir les positions de 64 cases) et β est encore élevé. Chaque simulation de RAVE informe sur bien plus de coups que MCTS — d'où une efficacité bien supérieure par simulation dans cette zone.

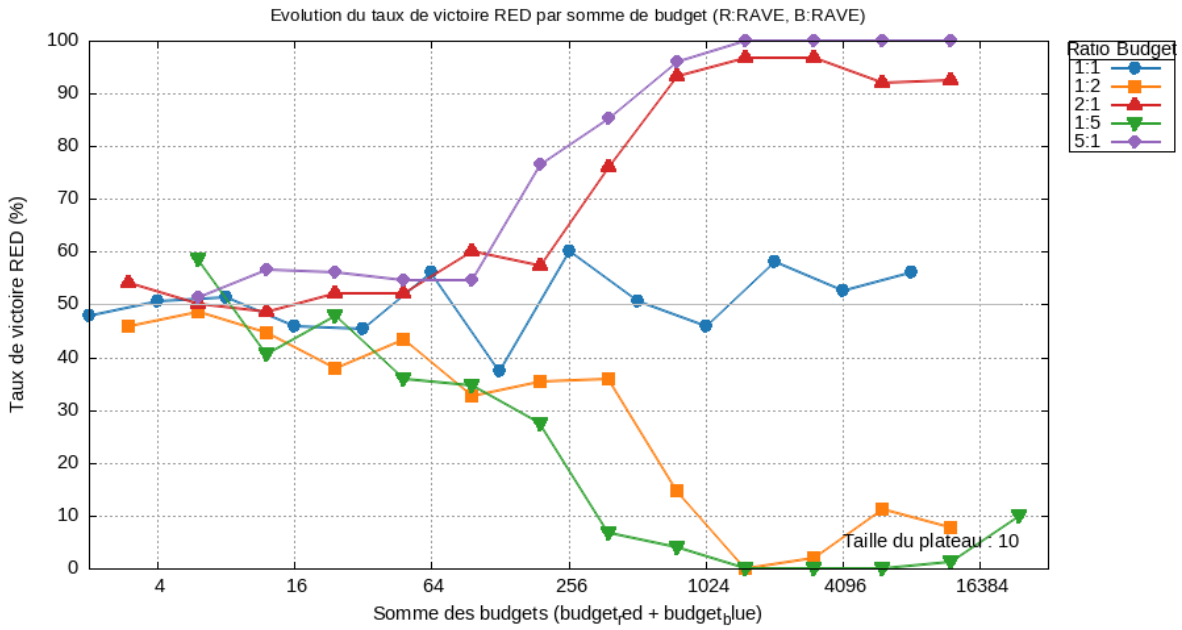


FIGURE 6.6. Taux de victoire RED · RAVE vs RAVE · Plateau 10×10 · 150 parties/point

Plateau 10×10 — 100 cases. Sur 100 cases, chaque simulation fournit ∼100 mises à jour AMAF. RAVE devient opérationnel à des budgets très faibles — les effets les plus précoces et les plus intenses de toute l'expérience.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 monte progressivement vers ∼55–58% aux grands budgets, dépassant l'IC95 ±8 pts. Avec MCTS taille 10, la courbe 1 :1 oscillait autour de 50% sans dépasser le seuil de significativité.	Sur 100 cases, les simulations couvrent ∼100 coups. Les coups d'ouverture de RED apparaissent dans une fraction très grande de chaque simulation et accumulent des statistiques AMAF particulièrement riches. RAVE révèle ici l'avantage du premier joueur que MCTS était incapable de détecter à cette taille.
La courbe 1 :5 atteint 0% très tôt — plus tôt qu'avec MCTS sur la même taille et plus tôt que RAVE sur taille 8. Elle reste strictement à 0% sur toute la plage suivante. Signal robuste (IC95 ±3,5 pts autour de 0%).	BLUE avec 5× plus de sim/coup construit un atlas AMAF tellement plus complet que ses statistiques couvrent des positions que RED n'a jamais vues. L'avantage du premier joueur est totalement écrasé dès ce seuil.

6.2.3 MCTS vs RAVE

Comportements communs aux trois tailles

Dans cette configuration, RED utilise MCTS et BLUE utilise RAVE. Trois comportements se retrouvent sur les trois tailles et révèlent la dynamique fondamentale entre les deux algorithmes.

■ Ce qu'on observe	■ Pourquoi
Sur les trois graphiques, les courbes ne sont pas monotones : RED monte fortement aux budgets intermédiaires, puis redescend aux très grands budgets. Ce comportement en cloche généralisé est présent pour presque tous les ratios — même le ratio 1 :1.	À budget intermédiaire, MCTS explore suffisamment l'arbre pour prendre de bonnes décisions, tandis que RAVE (BLUE) n'a pas encore accumulé assez de statistiques AMAF pour être fiable. MCTS domine donc dans cette fenêtre. Aux grands budgets, les statistiques AMAF de BLUE deviennent très riches et RAVE tire pleinement profit de son efficacité supérieure — BLUE reprend l'avantage.
La courbe 1 :1 monte au-dessus de 50% aux budgets intermédiaires, parfois jusqu'à 60–70%, puis redescend vers 50% aux grands budgets. C'est un comportement unique par rapport aux confrontations MCTS vs MCTS et RAVE vs RAVE où la courbe 1 :1 était plus stable.	À budget égal, MCTS et RAVE partent à égalité mais MCTS bénéficie d'abord du fait que RAVE n'a pas encore ses statistiques AMAF. RED (MCTS) obtient donc un avantage temporaire combiné à l'avantage du premier joueur. Quand RAVE devient opérationnel à grand budget, son efficacité supérieure compense — la courbe revient vers 50%.
Les courbes de MCTS vs RAVE et RAVE vs MCTS sont quasi-symétriques autour de 50%.	Intervertir les algorithmes de RED et BLUE inverse les rôles. Cette symétrie valide la cohérence du protocole et confirme que les effets observés sont bien dus aux algorithmes et non à des artefacts expérimentaux.

Point clé

Synthèse : MCTS (RED) domine à budget intermédiaire car RAVE (BLUE) n'a pas encore ses statistiques AMAF. Mais aux grands budgets, RAVE reprend l'avantage grâce à son efficacité supérieure par simulation. La taille du plateau détermine à quel budget ce basculement se produit.

Spécificités de chaque taille

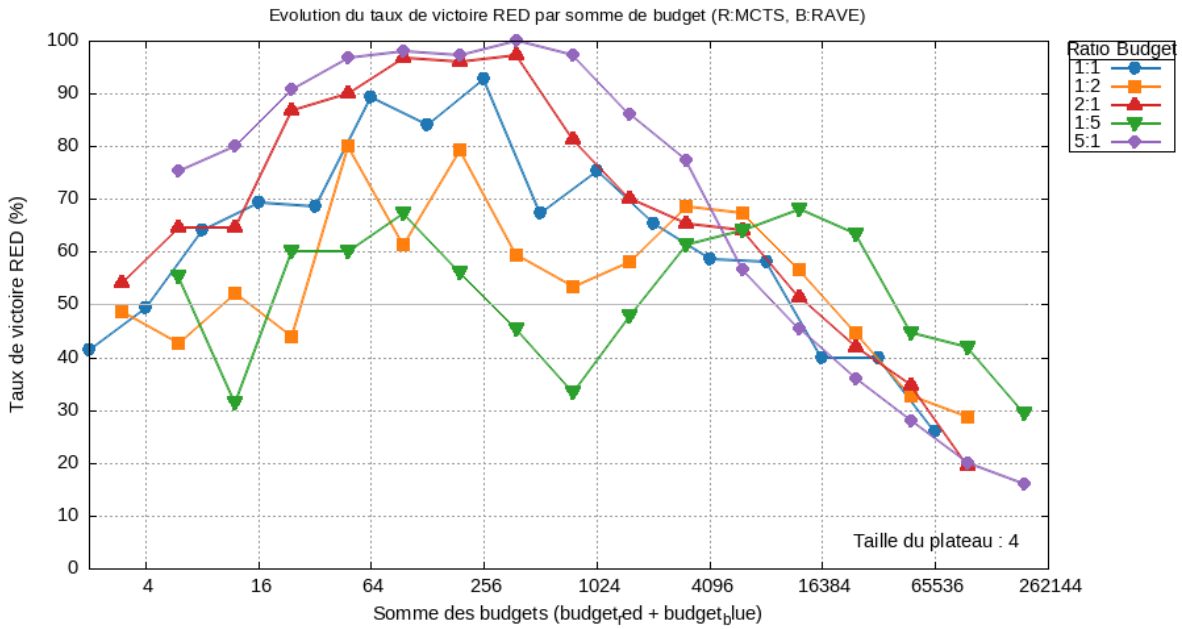


FIGURE 6.7. Taux de victoire RED · MCTS vs RAVE · Plateau 4×4 · 150 parties/point

Plateau 4×4 — 16 cases. Sur taille 4, la confrontation directe MCTS vs RAVE produit des comportements complexes, mêlant l'effet du ratio, la quasi-résolution du plateau, et la dynamique entre les deux algorithmes.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 monte progressivement jusqu'à ∼90% aux budgets intermédiaires — bien au-dessus de ce qu'on observait en MCTS vs MCTS (60–65%) et en RAVE vs RAVE (47–73%). Puis elle redescend nettement vers 50–55% aux très grands budgets.	L'avantage du premier joueur (RED) se combine avec l'avantage temporaire de MCTS sur RAVE aux budgets intermédiaires : RAVE n'a pas encore ses statistiques AMAF sur 16 cases. Aux grands budgets, RAVE devient opérationnel et efface cet avantage. La redescente vers 50% confirme que les deux algorithmes convergent quand le budget est suffisant.
Les courbes 2 :1 et 5 :1 montent très haut (95–100%) aux budgets intermédiaires, puis redescendent fortement aux grands budgets vers 60–70%. La redescente est plus prononcée qu'en MCTS vs MCTS seul.	RED cumule deux avantages : plus de sim/coup ET l'algorithme MCTS qui est temporairement plus efficace que RAVE. Aux grands budgets, RAVE de BLUE rattrape son retard algorithmique et la combinaison (budget supérieur + RAVE opérationnel) fait redescendre fortement les courbes.
La courbe 1 :5 s'effondre à 0% aux budgets intermédiaires puis remonte aux très grands budgets.	BLUE cumule deux avantages : plus de sim/coup ET RAVE qui devient opérationnel avant que MCTS de RED ait le temps de compenser. La remontée aux grands budgets s'explique par la quasi-résolution du plateau 4×4.

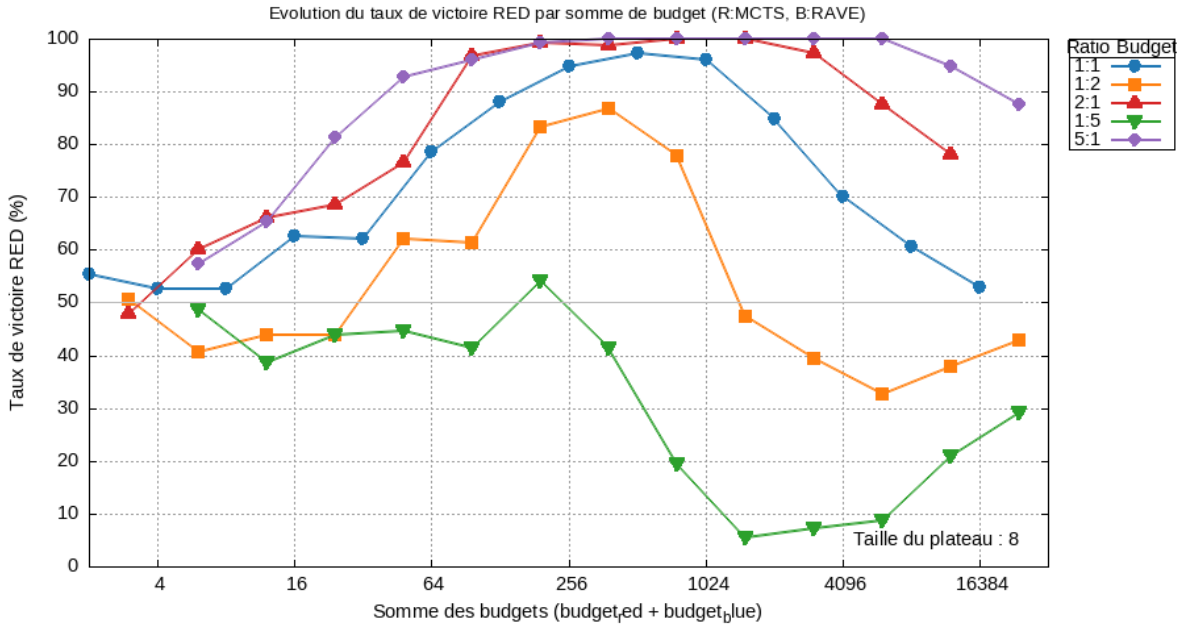


FIGURE 6.8. Taux de victoire RED · MCTS vs RAVE · Plateau 8×8 · 150 parties/point

Plateau 8×8 — 64 cases. Sur taille 8, le comportement en cloche est encore plus net. Le basculement entre la domination de MCTS et la reprise de RAVE est clairement visible sur toutes les courbes.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 est la plus remarquable : elle part de ∼55%, monte jusqu'à ∼95% aux budgets intermédiaires, puis redescend vers ∼53% aux grands budgets. Cet arc est bien plus prononcé que tout ce qu'on observait en MCTS vs MCTS ou RAVE vs RAVE en ratio 1 :1.	À budget égal sur 64 cases, MCTS bénéficie d'un avantage temporaire car RAVE n'a pas encore accumulé assez de statistiques AMAF pour être fiable sur un plateau aussi grand. Combiné à l'avantage du premier joueur de RED, cela crée une fenêtre de domination très nette. Aux grands budgets, RAVE de BLUE accumule des statistiques AMAF riches sur 64 cases et reprend l'avantage.
Les courbes 2 :1 et 5 :1 montent vers 95–100% aux budgets intermédiaires, puis redescendent vers 75–87% aux grands budgets. Même avec un ratio très favorable, MCTS ne maintient pas sa domination face à RAVE aux grands budgets.	RED cumule avantage budgétaire + avantage temporaire de MCTS sur RAVE. Mais aux grands budgets, RAVE de BLUE compense son désavantage budgétaire par son efficacité supérieure par simulation. L'écart se réduit mais ne s'inverse pas totalement car le ratio reste en faveur de RED.
La courbe 1 :5 s'effondre à 0% dès les budgets intermédiaires et reste très basse. Sur taille 8, contrairement à taille 4, la remontée aux grands budgets est absente ou très limitée.	BLUE cumule avantage budgétaire + RAVE opérationnel sur 64 cases. Sur taille 8, il n'y a pas de quasi-résolution pour permettre à RED de rattraper son retard. L'avantage du premier joueur de RED est totalement écrasé par cette double pression.

Synthèse

Sur taille 8, la fenêtre de domination de MCTS (RED) est très nette aux budgets intermédiaires — notamment en ratio 1 :1 où la courbe monte jusqu'à ∼95%. C'est la preuve la plus directe que MCTS est temporairement meilleur que RAVE quand AMAF n'est pas encore fiable. Le basculement se produit clairement quand RAVE accumule suffisamment de statistiques sur 64 cases.

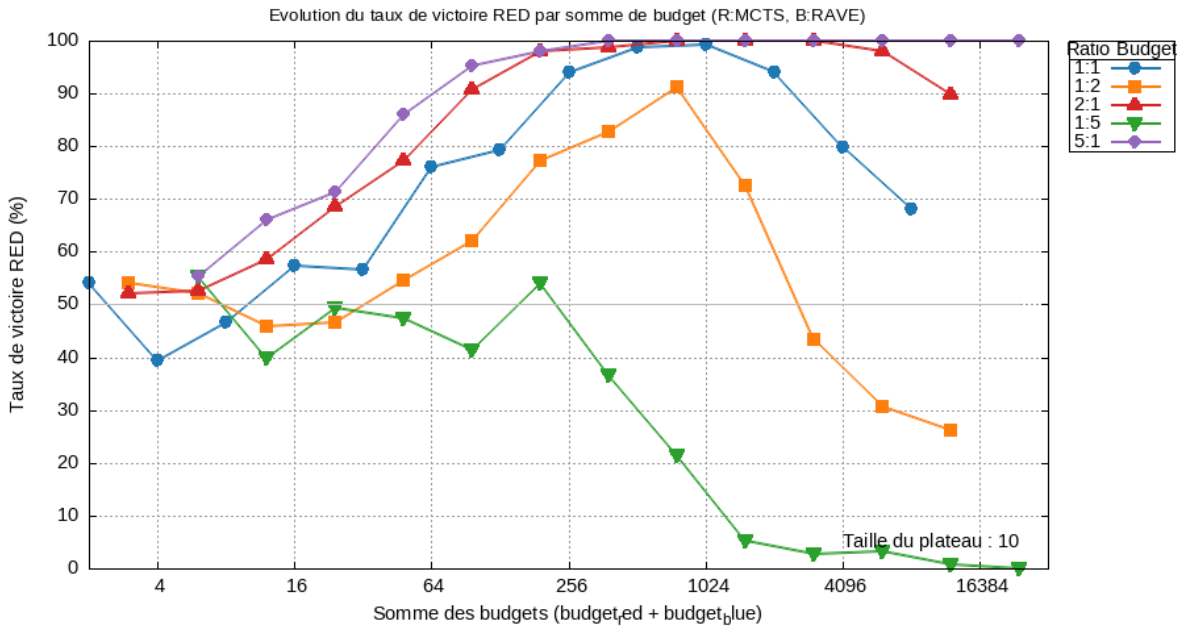


FIGURE 6.9. Taux de victoire RED · MCTS vs RAVE · Plateau 10×10 · 150 parties/point

Plateau 10×10 — 100 cases. Sur taille 10, la dynamique entre MCTS et RAVE est la plus lisible de l'expérience. Les simulations longues (∼100 coups) rendent RAVE opérationnel très tôt, ce qui raccourcit fortement la fenêtre de domination de MCTS.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 monte vers ∼75–80% aux budgets intermédiaires — moins haut qu'en taille 8 (∼95%) — puis redescend rapidement vers 50–55% aux grands budgets. La fenêtre de domination de MCTS est plus étroite qu'en taille 8.	Sur 100 cases, chaque simulation fournit ∼100 mises à jour AMAF. RAVE de BLUE devient opérationnel bien plus tôt qu'en taille 8, ce qui réduit la fenêtre où MCTS domine. L'avantage du premier joueur de RED est visible dans la montée initiale, mais RAVE rattrape rapidement.
Les courbes 2 :1 et 5 :1 montent vers 95–100% aux budgets intermédiaires, puis redescendent significativement aux grands budgets. La redescente est plus rapide et plus prononcée qu'en taille 8.	RAVE de BLUE devient opérationnel plus tôt sur taille 10 grâce aux simulations longues. La fenêtre où MCTS profite de son avantage temporaire est donc plus courte, et la reprise de RAVE se produit à des budgets moins élevés.
La courbe 1 :5 s'effondre à 0% très rapidement — encore plus tôt qu'en taille 8 — et reste à 0% sur toute la plage. Même la courbe 1 :2 descend vers des niveaux très bas.	BLUE cumule désavantage budgétaire de RED + RAVE opérationnel très tôt sur 100 cases. L'avantage du premier joueur de RED est totalement absorbé. Il n'y a pas de quasi-résolution pour permettre une remontée.

Synthèse

Sur taille 10, la fenêtre de domination de MCTS (RED) est plus courte qu'en taille 8 car RAVE devient opérationnel plus rapidement grâce aux simulations longues. C'est la preuve que la taille du plateau détermine directement à quel budget RAVE prend l'avantage sur MCTS.

6.2.4 RAVE vs MCTS

Comportements communs aux trois tailles

Dans cette configuration, RED utilise RAVE et BLUE utilise MCTS. C'est le miroir exact de la section précédente.

■ Ce qu'on observe	■ Pourquoi
Sur les trois graphiques, les courbes présentent un creux aux budgets intermédiaires — RED (RAVE) est dominé dans cette fenêtre — puis remontent aux grands budgets. C'est l'inverse exact du comportement observé en MCTS vs RAVE.	À budget intermédiaire, RAVE n'a pas encore accumulé assez de statistiques AMAF — MCTS de BLUE est temporairement plus efficace. Aux grands budgets, RAVE (RED) accumule des statistiques riches et son efficacité supérieure par simulation lui permet de reprendre l'avantage sur MCTS.
Pour un même ratio, les courbes de RAVE vs MCTS sont quasi-symétriques par rapport à 50% avec les courbes de MCTS vs RAVE.	Intervertir les algorithmes de RED et BLUE inverse exactement les rôles. Cette symétrie est une validation forte de la cohérence des expériences.

Point clé

Synthèse : RAVE (RED) est désavantagé à budget intermédiaire car ses statistiques AMAF ne sont pas encore fiables. Il reprend l'avantage aux grands budgets grâce à son efficacité supérieure par simulation. Plus la taille est grande, plus ce basculement se produit tôt.

Spécificités de chaque taille

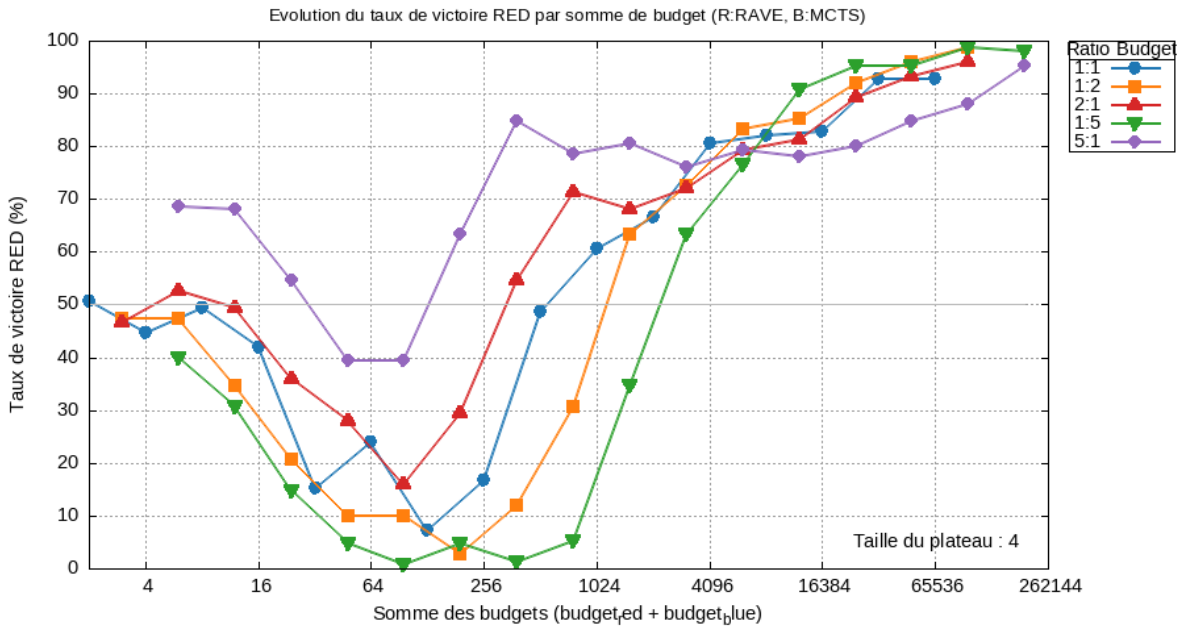


FIGURE 6.10. Taux de victoire RED · RAVE vs MCTS · Plateau 4×4 · 150 parties/point

Plateau 4×4 — 16 cases. Sur taille 4, RAVE vs MCTS confirme exactement la dynamique observée en miroir dans MCTS vs RAVE. Les comportements sont ici moins volatils car RED (RAVE) est le joueur qui doit attendre que ses statistiques AMAF deviennent fiables pour performer.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 reste autour de 45–55% aux petits budgets, oscille dans cette zone aux budgets intermédiaires, puis monte progressivement vers 60–70% aux grands budgets. C'est le miroir exact de la courbe 1 :1 en MCTS vs RAVE.	RAVE (RED) commence à la même puissance que MCTS mais sans statistiques AMAF — aucun avantage temporaire. L'avantage du premier joueur de RED maintient la courbe légèrement au-dessus de 50% même dans la zone défavorable. Aux grands budgets, RAVE accumule ses statistiques et reprend l'avantage sur MCTS.
Les courbes 2 :1 et 5 :1 présentent un creux aux budgets intermédiaires avant de remonter aux grands budgets. En MCTS vs MCTS, ces courbes étaient monotonement croissantes.	À budget intermédiaire, l'avantage budgétaire de RED est partiellement compensé par le fait que MCTS de BLUE est temporairement plus efficace que RAVE de RED faute de statistiques AMAF. Aux grands budgets, RAVE de RED devient opérationnel et son efficacité supérieure s'ajoute à l'avantage budgétaire — les courbes remontent et dépassent finalement les niveaux de MCTS vs MCTS.
Les courbes 1 :2 et 1 :5 s'effondrent aux budgets intermédiaires puis remontent progressivement aux très grands budgets, plus haut qu'en MCTS vs MCTS sur la même taille.	La remontée est due à deux effets combinés : la quasi-résolution du plateau 4×4 aux grands budgets, ET l'efficacité supérieure de RAVE qui permet à RED de rattraper davantage son désavantage budgétaire que MCTS ne le ferait dans la même situation.

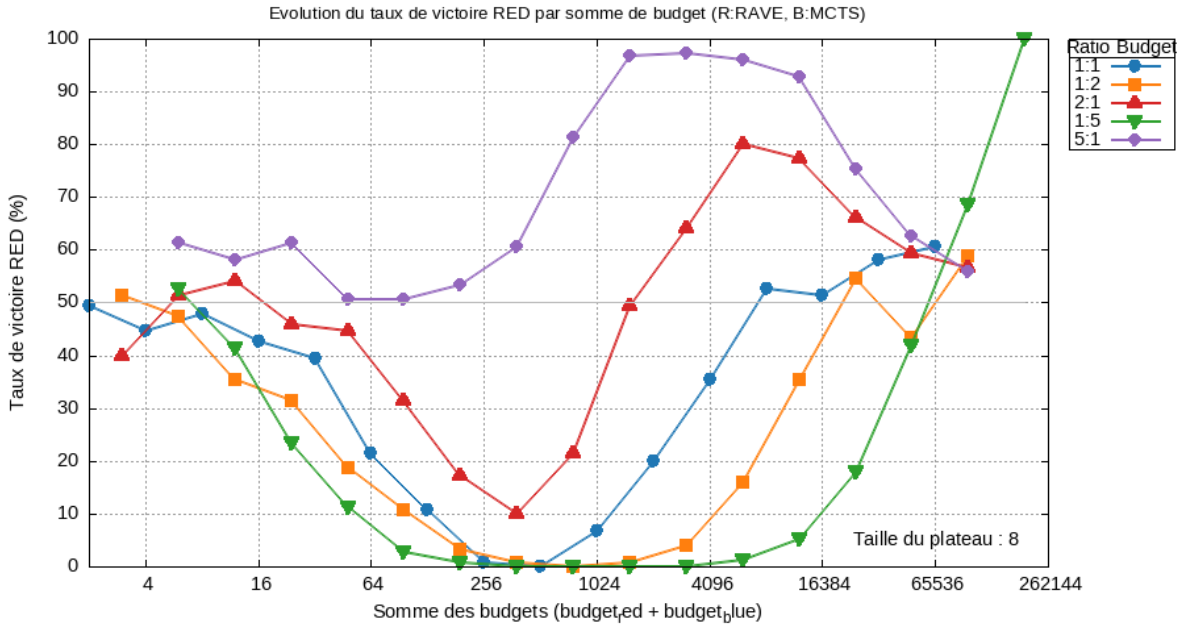


FIGURE 6.11. Taux de victoire RED · RAVE vs MCTS · Plateau 8×8 · 150 parties/point

Plateau 8×8 — 64 cases. Sur taille 8, RAVE vs MCTS est le miroir le plus lisible de MCTS vs RAVE. Le creux aux budgets intermédiaires et la reprise aux grands budgets sont clairement identifiables sur toutes les courbes.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 part de ∼55%, descend vers ∼8% aux budgets intermédiaires, puis remonte vers 55–60% aux grands budgets. C'est l'inverse exact de la courbe 1 :1 en MCTS vs RAVE qui montait jusqu'à ∼95% puis redescendait.	Le creux correspond exactement à la fenêtre où MCTS de BLUE est temporairement plus efficace que RAVE de RED. L'avantage du premier joueur de RED maintient la courbe au-dessus de 0% même dans le creux — RED gagne encore ∼8% des parties grâce à cet avantage structurel. La remontée aux grands budgets confirme que RAVE reprend l'avantage quand ses statistiques AMAF sont fiables sur 64 cases.
Les courbes 2 :1 et 5 :1 présentent un creux prononcé aux budgets intermédiaires avant de remonter fortement aux grands budgets. La remontée finale dépasse les niveaux atteints en MCTS vs MCTS pour les mêmes ratios.	À budget intermédiaire, MCTS de BLUE compense partiellement son désavantage budgétaire par son efficacité temporaire. Aux grands budgets, RAVE de RED cumule avantage budgétaire ET efficacité supérieure — les courbes remontent au-delà de ce que MCTS vs MCTS permettrait avec les mêmes ratios.
La courbe 1 :5 s'effondre à 0% aux budgets intermédiaires puis remonte légèrement aux grands budgets — contrairement à MCTS vs MCTS taille 8 où il n'y avait aucune remontée.	La légère remontée aux grands budgets, absente en MCTS vs MCTS, est due à l'efficacité supérieure de RAVE qui permet à RED de partiellement compenser son désavantage budgétaire. RAVE récupère mieux que MCTS dans la même situation défavorable.

Synthèse

Sur taille 8, RAVE vs MCTS confirme la dynamique en miroir : le creux en ratio 1 :1 (∼8%) correspond exactement au pic de MCTS vs RAVE (∼95%). La reprise de RAVE aux grands budgets est robuste et démontre son efficacité supérieure par simulation une fois les statistiques AMAF fiables.

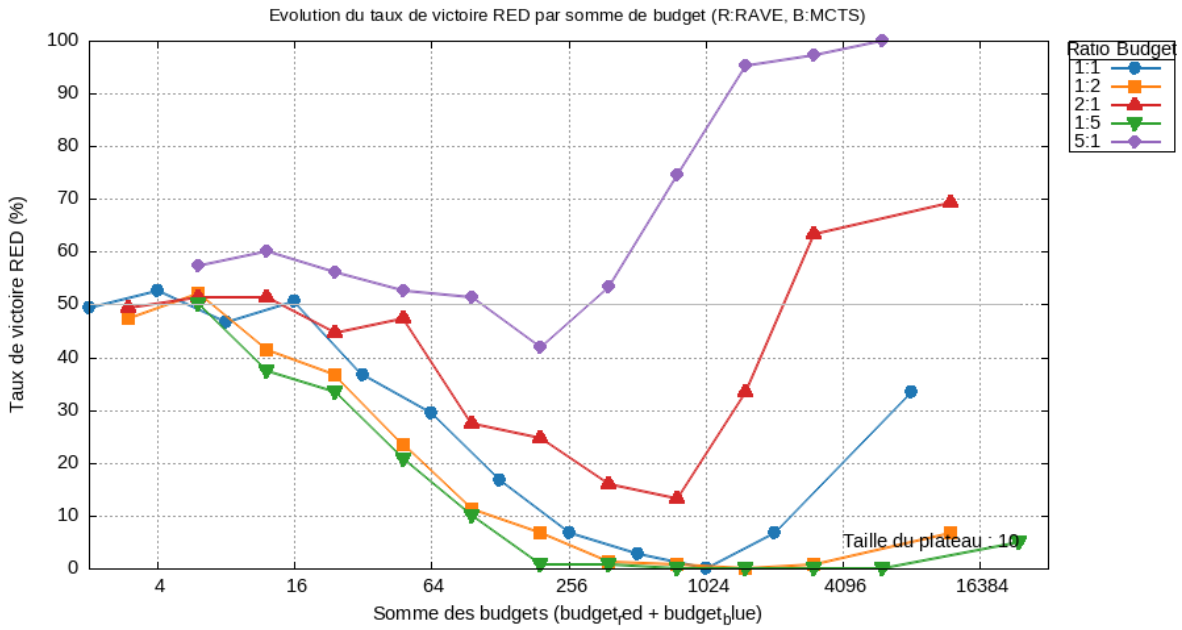


FIGURE 6.12. Taux de victoire RED · RAVE vs MCTS · Plateau 10×10 · 150 parties/point

Plateau 10×10 — 100 cases. Sur taille 10, les simulations longues (∼100 coups) accélèrent la disponibilité des statistiques AMAF — RAVE devient opérationnel plus tôt, ce qui raccourcit le creux et accélère la reprise.

■ Ce qu'on observe	■ Pourquoi
La courbe 1 :1 présente un creux moins profond et plus court qu'en taille 8 avant de remonter vers 55–60% aux grands budgets. La reprise de RAVE se produit à des budgets plus faibles que sur taille 8.	Sur 100 cases, les simulations de ∼100 coups enrichissent rapidement les statistiques AMAF de RAVE. Le désavantage temporaire de RAVE face à MCTS est donc plus court qu'en taille 8. L'avantage du premier joueur de RED aide RAVE à maintenir un niveau minimal même dans le creux.
Les courbes 2 :1 et 5 :1 montent vers 95–100% aux grands budgets après un creux aux budgets intermédiaires. La reprise finale est la plus haute de toute l'expérience pour des ratios favorables en RAVE vs MCTS.	Sur 100 cases, RAVE accumule ses statistiques très vite et son efficacité est maximale. Une fois opérationnel, RED cumule avantage budgétaire + RAVE pleinement efficace contre MCTS de BLUE — combinaison qui produit les niveaux les plus élevés de l'expérience.
La courbe 1 :5 s'effondre à 0% aux budgets intermédiaires puis remonte vers 30–35% aux grands budgets — une remontée bien plus forte qu'en MCTS vs MCTS taille 10 où la courbe restait à 0% sans jamais remonter.	C'est la démonstration la plus claire de l'avantage de RAVE sur MCTS en situation défavorable : même avec 5× moins de budget, RAVE parvient à remonter vers 30–35% aux grands budgets grâce à son efficacité supérieure par simulation. MCTS dans la même situation ne remontait jamais.

Synthèse

Sur taille 10, RAVE vs MCTS produit la démonstration la plus convaincante de l'efficacité supérieure de RAVE : même en ratio défavorable 1 :5, RAVE finit par remonter vers 30–35% là où MCTS restait à 0%. La taille du plateau, en enrichissant rapidement les statistiques AMAF, permet à RAVE d'exploiter pleinement son avantage algorithmique.

6.2.5 Conclusion de l'analyse expérimentale

1. Le **ratio budgétaire par coup** est le facteur dominant sur toutes les configurations.
2. L'**avantage du premier joueur** est réel mais subordonné au ratio et à la taille (visible sur 4×4, non significatif sur 10×10 avec MCTS).
3. La **taille du plateau** accélère les effets de ratio et dilue l'avantage du premier joueur.
4. **RAVE est plus efficace par simulation** (AMAF) et amplifie les trois facteurs. À budget intermédiaire il surpasse MCTS ; à grand budget les deux convergent.
5. La **quasi-résolution du plateau 4×4** est un phénomène unique : elle atténue les effets de ratio aux grands budgets et maintient un plancher pour les ratios défavorables.

6.3 Critère de sélection du meilleur enfant

Dans notre implémentation de MCTS, la sélection finale du meilleur coup (méthode `getBestMove`) est basée sur le **nombre de visites** du nœud enfant. Ce critère est généralement préféré au ratio victoires/visites car il est moins sensible aux fluctuations statistiques dans les nœuds peu explorés : un nœud très visité a tendance à avoir une estimation plus fiable de sa valeur réelle.

Notre implémentation dispose également d'une méthode `getBestRatio` sélectionnant l'enfant avec le meilleur ratio victoires/visites, mais celle-ci n'est pas utilisée dans les expérimentations. Une comparaison entre les deux critères constituerait une piste d'amélioration intéressante.

6.4 Paramètre d'exploration c

Le paramètre c de la formule UCB contrôle l'équilibre entre exploration et exploitation :

$$\text{UCB}(n) = \frac{v_n}{V_n} + c \cdot \sqrt{\frac{\ln V_{\text{parent}}}{V_n}}$$

Dans notre implémentation, c est fixé à la valeur standard $c = \sqrt{2}$, conformément aux recommandations de la littérature [**?**]. Cette valeur offre un bon équilibre entre exploration et exploitation pour les tailles de plateau testées (4×4, 8×8, 10×10). Une étude de sensibilité sur ce paramètre constituerait une perspective d'expérimentation future.

7. Reverse Hex (Misère Hex)

7.1 Présentation de la variante

Le **Reverse Hex**, aussi appelé **Misère Hex**, est une variante du jeu de Hex dans laquelle la règle de victoire est inversée : le joueur qui relie ses deux bords opposés **perd** la partie. Les deux joueurs cherchent donc à *éviter* de compléter leur propre chemin tout en forçant l'adversaire à compléter le sien.

Règle du Reverse Hex

- **Rouge** perd s'il relie le bord Nord au bord Sud.
- **Bleu** perd s'il relie le bord Ouest au bord Est.
- Le gagnant est donc celui qui **force l'autre** à compléter sa connexion en premier.

Cette inversion transforme profondément la stratégie : là où le Hex classique récompense la connexion agressive, le Reverse Hex exige de jouer de manière défensive et de pousser l'adversaire à occuper des cases qui l'obligeront à se connecter.

7.2 Adaptations techniques

7.2.1 Modification de la condition de victoire

L'adaptation principale porte sur la méthode `isAWin()` de la classe `Board`. Dans le Hex classique, un joueur gagne s'il a lui-même relié ses bords. Dans le Reverse Hex, un joueur *c* gagne si son **adversaire** a relié ses propres bords. La détection DFS reste identique, mais elle est appliquée à la couleur opposée :

Logique de victoire inversée dans `Board.java`

- Dans le Hex classique : `isAWin(c)` retourne `true` si *c* a relié ses bords.
- Dans le Reverse Hex : `isAWin(c)` retourne `true` si l'**adversaire** de *c* a relié ses bords.

En pratique, le DFS est donc appliqué sur la couleur `autre = opposé(c)`, en vérifiant ses bords `bordsAutre`.

7.2.2 Modification de la simulation dans MCTS et RAVE

La méthode de simulation (*playout*) doit également évaluer la victoire selon la règle inversée. Une méthode dédiée `isAWinInversed(c, grid)` a été ajoutée dans les classes `MCTS` et `RAVE` :

Méthode	Rôle
`isAWin(grid)`	Détecte si la partie est terminée (l'un des deux joueurs a connecté ses bords)
`isAWinInversed(c, grid)`	Retourne `true` si le joueur *c* a **gagné** selon la règle misère

Ainsi, l'arbre de recherche est guidé non plus vers les coups qui construisent une connexion, mais vers les coups qui *forcent l'adversaire* à en construire une.

7.2.3 Résumé des modifications

Les modifications apportées au code sont minimales et localisées, ce qui illustre la modularité de l'architecture en packages :

- `Board.java` : inversion de la logique dans `isAWin()`.
- `MCTS.java` : ajout de `isAWinInversed()` utilisée dans `simulation()`.
- `RAVE.java` : même ajout de `isAWinInversed()` dans `simulation()`.
- `Game.java` : adaptation de `getWinner()` pour retourner le bon vainqueur selon la règle misère.

Le reste du code — la structure de l'arbre, les formules UCT et RAVE, la rétropropagation, le DFS

— est **inchangé**.

Le diagramme de classes du Reverse Hex est identique à celui du Hex classique présenté au chapitre précédent. Les mêmes packages `Game`, `Mcts` et `Rave` sont utilisés, avec les mêmes classes et les mêmes relations entre elles. Seule la logique interne de quelques méthodes a été modifiée, sans qu'aucune nouvelle classe ne soit ajoutée.

7.3 Résultats de test

Les tests ont été menées sur un plateau 8×8 avec un budget de 1 000 simulations par coup. Les quatre configurations ont été testées et les temps de match mesurés :

Configuration	Premier joueur	Temps moyen	Complexité
MCTS vs MCTS	**Rouge**	24,07 s	Standard
RAVE vs RAVE	**Rouge**	25,02 s	Élevée (AMAF)
MCTS vs RAVE	**Rouge**	24,47 s	Hybride
RAVE vs MCTS	**Rouge**	25,04 s	Hybride

Les quatre configurations affichent des temps très proches, compris entre 24 et 25 secondes. MCTS seul est le plus rapide (24,07 s) car il ne maintient pas de statistiques AMAF. Les configurations impliquant RAVE sont légèrement plus lentes ($\sim$1 s d'écart) en raison du surcoût des mises à jour AMAF à chaque rétropropagation. Ces résultats confirment que l'inversion de la règle de victoire propre au Reverse Hex n'engendre aucun surcoût algorithmique supplémentaire par rapport au Hex classique.

7.4 Analyse et intérêt de la variante

Le Reverse Hex illustre un aspect important de la conception logicielle : une bonne architecture modulaire permet d'adapter un programme à une variante de problème avec un minimum de modifications. L'ensemble des algorithmes MCTS et RAVE a pu être réutilisé sans restructuration, grâce à la séparation claire entre la logique du jeu (`Game`), la recherche (`Mcts`) et l'optimisation (`Rave`).

Sur le plan théorique, le Reverse Hex est connu pour être un jeu où la stratégie optimale est plus complexe à identifier que dans le Hex classique. En particulier, l'avantage du premier joueur, prouvé dans le Hex classique, ne s'applique plus de la même façon dans la variante misère.

8. Conclusion

Ce projet nous a permis d'implémenter et de comparer les algorithmes MCTS et RAVE sur le jeu de Hex, en étudiant l'influence du rapport de budget sur les résultats. Trois conclusions principales se dégagent.

Le **ratio budgétaire par coup** est le facteur dominant : un fort déséquilibre écrase l'avantage du premier joueur de façon irréversible sur les plateaux 8×8 et 10×10. **RAVE surpasse MCTS** à budget intermédiaire grâce à l'heuristique AMAF, qui extrait plus d'information par simulation ; les deux algorithmes convergent aux très grands budgets. Enfin, la **taille du plateau** accélère les effets de ratio et dilue l'avantage structurel du premier joueur.

L'implémentation du **Reverse Hex** a démontré la modularité de l'architecture : seule la condition de victoire a été modifiée, sans restructuration des algorithmes.

Ce travail ouvre la voie à plusieurs pistes : tester des plateaux plus grands, étudier la sensibilité au paramètre c, approfondir l'analyse du Reverse Hex, ou confronter nos agents à MoHex.

Nous remercions M. Gregory Bonnet pour son encadrement tout au long de ce projet.

Références

1. Browne, C. *Hex Strategy : Making the Right Connections*. A K Peters/CRC Press, 2000.
2. Coulom, R. *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. Computers and Games, LNCS 4630, 2006.
3. Kocsis, L. & Szepesvári, C. *Bandit based Monte-Carlo Planning*. ECML, 2006.
4. Gelly, S. & Silver, D. *Combining Online and Offline Knowledge in UCT*. ICML, 2007.
5. Hayward, R. *A Thesis on Hex*. Université de l'Alberta.
   `https://webdocs.cs.ualberta.ca/~hayward/theses/ph.pdf`
6. Wikipedia. *Hex (jeu de plateau)*. `https://fr.wikipedia.org/wiki/Hex`